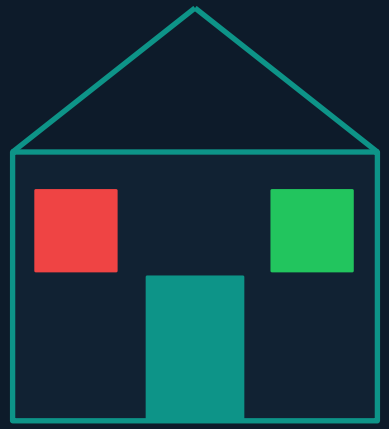
CS5272 · Embedded Software Design

Privacy-First Smart Home Control with Edge LLMs

Natural language commands processed locally — no cloud, no data leaks



Raspberry Pi 5

Zhao Haochen · E1553606 · e1553606@u.nus.edu

github.com/HaochenZhao/edgeLLM

Why Build a Private Smart Home?

01

Your Voice Leaves Home

Alexa, Google Home, and similar systems stream every command to remote servers. Your daily routines, schedules, and habits become third-party data.

02

Network Dependency

Internet outage = smart home becomes dumb. Cloud-dependent systems fail completely when connectivity is lost.

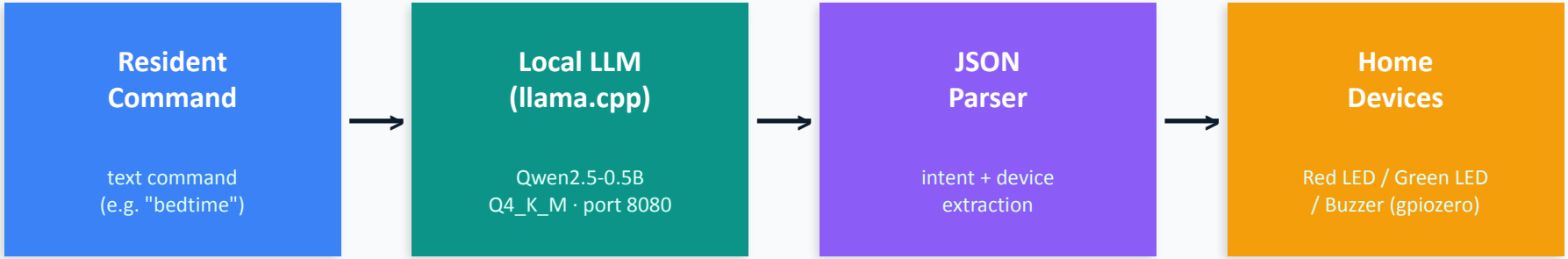
03

Rigid Trigger Phrases

Say "lights off" — it works. Say "I'm heading to bed" — it fails. Rule engines cannot interpret natural, contextual human language.

Solution: Run a quantized LLM on a Raspberry Pi 5 — no internet required. Vague commands are understood locally and mapped directly to home device control.

System Architecture



All processing happens on-device. No data ever leaves the home network.

Simulated Home Devices (GPIO)

- Red LED (GPIO 17) — Living room light
- Green LED (GPIO 27) — Bedroom light
- Buzzer (GPIO 23) — Doorbell / alarm

Software Stack

- llama.cpp — ARM NEON-optimized inference
- Qwen2.5-0.5B-Instruct (Q4_K_M, ~400MB)
- Python middleware — REST -> JSON -> GPIO
- gpiozero — hardware abstraction layer

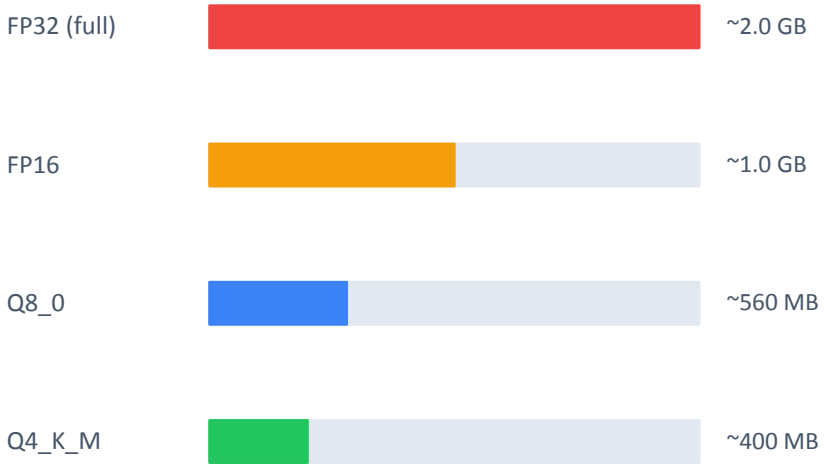
LLM Inference on the Edge: llama.cpp & Quantization

llama.cpp

- C++ inference engine — no Python runtime overhead
- ARM NEON SIMD intrinsics exploit the RPi 5's Cortex-A76 cores for vectorised matrix multiply
- Serves an OpenAI-compatible REST API (llama-server) on port 8080 — Python calls it via HTTP POST
- Cross-compiled directly on the Pi 5; no host machine needed
- Model loaded once at startup; subsequent queries are stateless HTTP requests

```
$ llama-server -m qwen2.5-0.5b-instruct-q4_k_m.gguf --port
8080
```

4-bit Quantization (Q4_K_M)



- Q4_K_M: weights stored in 4-bit integers, grouped with k-quant mixed precision
- Accuracy loss vs FP32 is negligible for instruction-following tasks
- GGUF format: single-file, memory-mapped — fast cold start
- Model: Qwen2.5-0.5B-Instruct pulled from Hugging Face Hub

Hardware Setup: Raspberry Pi 5 & GPIO Wiring

Breadboard Wiring

Device	GPIO (BCM)	Phys. Pin	Resistor
Red LED (living room)	GPIO 17	Pin 11	220 ohm
Green LED (bedroom)	GPIO 27	Pin 13	220 ohm
LED Common (GND)	GND	Pin 6/9	None
Buzzer (+)	GPIO 23	Pin 16	None
Buzzer (-)	GND	Pin 14/20	None

gpiozero Driver (hardware_control.py)

```
from gpiozero import LED, Buzzer
living_room = LED(17)   # Red LED
bedroom     = LED(27)   # Green LED
buzzer      = Buzzer(23)

def control(device, action, duration=1):
    if device == 'living_room_led':
        living_room.on() if action=='on' else living_room.off()
```

Raspberry Pi 5 Specs

- CPU: Cortex-A76 quad-core @ 2.4 GHz
- RAM: 8 GB LPDDR4X
- GPIO: 40-pin header (3.3V logic)
- OS: Raspberry Pi OS (64-bit)
- Power: 27W official PSU

Software Pipeline: LLM Interface & JSON Parser

main.py — Control Loop

1 **Read input**
User types NL command in terminal

>

2 **HTTP POST**
Send prompt to llama-server /v1/chat/completions

>

3 **Parse JSON**
Extract device & action from LLM response

>

4 **GPIO call**
Call hardware_control.py function

>

5 **Log & loop**
Print result, await next command

llm_interface.py — LLM call

```
import requests, json

SYSTEM = "You control a smart home."
         "Output ONLY valid JSON."

def ask_llm(user_cmd):
    r = requests.post(
        'http://localhost:8080/v1/chat/completions',
        json={ 'messages': [
            {'role':'system','content':SYSTEM},
            {'role':'user',  'content':user_cmd}
        ], 'temperature': 0.1 })
    return r.json()['choices'][0]
             ['message']['content']
```

main.py — JSON parser & error handling

```
def run_command(user_cmd):
    raw = ask_llm(user_cmd)
    try:
        cmd = json.loads(raw)
        device = cmd['device']
        action = cmd['action']
        duration = cmd.get('duration', 1)
        control(device, action, duration)
    except (json.JSONDecodeError,
            KeyError) as e:
        print(f'Parse error: {e}')
        print(f'Raw output: {raw}')
        # skip GPIO — safe fallback
```

Natural Language -> Home Device Control

Command Examples

User: *"I'm heading to bed"*

LLM: `{"device":"bedroom_led","action":"on"}`

Green LED on (Bedroom light)

User: *"Living room off, good night"*

LLM: `{"device":"living_room_led","action":"off"}`

Red LED off (Living room light)

User: *"Someone is at the door"*

LLM: `{"device":"buzzer","action":"beep","duration":2}`

Buzzer beeps (Doorbell alert)

Robust parser catches malformed JSON — logs error and skips the GPIO call safely.

System Prompt

```
You control a smart home.
Output ONLY valid JSON.

Devices:
 living_room_led
   actions: on, off, dim
 bedroom_led
   actions: on, off, dim
 buzzer
   actions: beep, off
   duration: (seconds)

No explanation.
No extra text.
JSON only.
```

Expected Results (Benchmarking In Progress)

Benchmarking is currently ongoing — figures below are targets based on llama.cpp documentation and preliminary tests.

< 3s	~18	~400MB	> 90%
End-to-end latency target	Tokens/sec target RPi 5 (no GPU)	RAM footprint Q4_K_M model	Expected device control accuracy

Metrics Being Measured

- Time To First Token (TTFT) — latency from HTTP request to first byte of response
- Generation throughput (tokens/sec) — measured across 50 varied NL home commands
- RAM utilisation — monitored via /proc/meminfo during sustained inference
- CPU thermal throttling — core temperature logged under continuous 5-min load
- Intent accuracy — whether parsed JSON device/action matches expected output

Future Work

- Offline voice input via Whisper.cpp
- Expand to more rooms and devices (servo, relay)
- Cross-session context memory
- PWM dimming for LEDs via gpiozero PWMLED
- Automated test harness for accuracy scoring

Conclusion

- Privacy by design — commands processed on-device, never sent to any server
- llama.cpp + Q4_K_M quantization makes a 0.5B-param LLM viable on a cheap CPU board
- JSON prompting replaces brittle rule engines with flexible natural language understanding
- Works fully offline — living room light, bedroom light, and doorbell respond during outages

Thank you! Questions welcome